

Hidden Gem Music

Team Reference Notes

Architecture · Database Schema · Stored Procedures · KPI Definitions · Filters · Recharts · Design Decisions

SOFT290 Capstone | Leena, Eli, Landin | April 2026 | Living document — update as decisions are made

1. The Discovery Gap — North Star for Every Decision

Every screen, every filter, every chart, every KPI, and every stored procedure exists to answer one question: which globally trending artists and songs are missing from a given market, and how do those patterns change over time?

The discovery gap is not a feature — it is the entire point of the app. Any design or data decision that does not clearly serve that question should be challenged. When in doubt about what to show, what to filter, or what to compute: whatever most directly surfaces the gap wins.

This framing must be visible to the user at every level of the app:

- Splash / Welcome Screen — introduces the problem explicitly before the user touches anything. Includes the data disclaimer about the 22-month gap between datasets.
- Globe — visualizes the gap geographically: where is music spreading, and where has it not arrived yet
- Filters — not data-narrowing tools, but lenses for exploring a specific version of the gap
- Country Profile Page — what is this country missing, and how does that compare to what it has
- Comparison Page — how does the gap differ between two markets, and what are both of them missing
- Hidden Gems Screen — the most direct expression of the gap: songs trending elsewhere that have not crossed over here
- Global Overlap Dashboard — the analytical answer: the gap quantified, trended over time, broken down globally via four headline KPIs

Every chart and KPI on every page should be legible as an answer to some version of the discovery gap question. If a chart cannot be explained in terms of the gap, question whether it belongs.

2. Data Sources

The schema unifies two Kaggle datasets that overlap in subject matter but differ in structure, geographic encoding, chart scope, and column naming. They are never directly compared where their metrics differ.

Dataset 1 — Spotify Historical Charts (2017–2021)

- Approx. 3.5GB — estimated 25–30 million raw rows
- Covers Spotify Top 200 and Viral 50 charts published every 2–3 days, Jan 1 2017 – Dec 30 2021
- Countries stored as full names (e.g. "United States") — bridged to ISO codes via Country lookup table
- Songs identified by Spotify URL — track ID parsed from URL during cleaning to populate `spotify_id`
- `streams` is NULL for all Viral 50 entries
- trend column values: `MOVE_UP`, `MOVE_DOWN`, `SAME`, `NEW_ENTRY`
 - Normalized during population: `MOVE_UP` = +1, `MOVE_DOWN` = -1, `SAME` = 0, `NEW_ENTRY` = signal flag
 - `NEW_ENTRY` is meaningful for Hidden Gem scoring — a song's first appearance in a country is a strong signal
- Source columns: `title`, `rank`, `date`, `artist`, `url`, `region`, `chart`, `trend`, `streams`

Dataset 2 — Top Spotify Songs in 73 Countries (2023–2025)

- Approx. 500MB — estimated 3–5 million rows
- Covers Top 50 songs per country updated daily, Oct 17 2023 – Jun 10 2025
- Countries stored as 2-letter ISO codes (e.g. "US")
- Songs identified by native `spotify_id` — this is the cross-dataset matching key
- Includes full audio feature data: `danceability`, `energy`, `tempo`, `valence`, `speechiness`, `acousticness`, `instrumentalness`, `liveness`, `loudness`, `key`, `mode`, `time_signature`
- `daily_movement` and `weekly_movement` are actual numeric integer deltas — more precise than Dataset 1's categorical trend strings
- Some rows have NULL country — these are Global Top 50 entries, stored with `country_id` = NULL in `ChartEntry`
- `artists` column is a comma-separated string — split on `,` during cleaning to produce individual Artist rows and ArtistSong join rows
- Source columns: `spotify_id`, `name`, `artists`, `daily_rank`, `daily_movement`, `weekly_movement`, `country`, `snapshot_date`, `popularity`, `is_explicit`, `duration_ms`, `album_name`, `album_release_date` + 13 audio feature columns

Critical Cross-Dataset Notes

- Audio features (`danceability`, `energy`, `valence`, `tempo`, etc.) exist in Dataset 2 only — this means NO external API is needed for audio feature enrichment. Eli's external API plan, if any, is only relevant for genre tags. Clarify before scoping genre filters.
- Popularity metrics must NEVER be compared directly: Dataset 1 uses raw stream counts, Dataset 2 uses Spotify's proprietary 0–100 popularity score.
- Song matching bridge: parse Spotify track ID from Dataset 1 URL. Not all will parse cleanly — unmatched songs get separate Song rows with NULLs for missing columns.
- Dataset 1 trend is a string category. Dataset 2 movement is a numeric integer. Both are stored in `ChartEntry` as separate nullable columns — they are never merged into a single column.

- 22-month data gap: Dec 2021 – Oct 2023. Every time-series visualization must visually mark this gap. Document on Welcome Screen data disclaimer.
- ChartEntry is estimated at 13–16 million rows combined. This is the primary performance concern for every stored procedure.

3. Confirmed Database Schema

Schema is fully normalized to 3NF. All heavy computation runs in stored procedures server-side. The API layer receives only clean aggregated results. Table names below are final.

Lookup Tables

Country

Bridges Dataset 1 full country names with Dataset 2 ISO codes. Add region column here for globe filter support.

Column	Type	Constraints	Description
country_id	INT	PK, IDENTITY	Auto-incremented surrogate primary key
full_name	NVARCHAR(100)	NOT NULL, UNIQUE	Full country name as used in Dataset 1 (e.g. 'United States')
iso_code	CHAR(2)	NOT NULL, UNIQUE	2-letter ISO code as used in Dataset 2 (e.g. 'US')
region	NVARCHAR(50)	NOT NULL	Geographic region for globe filter (e.g. 'Europe', 'Latin America', 'Southeast Asia')
latitude	FLOAT	NULL	Latitude coordinate for Mapbox tileset
longitude	FLOAT	NULL	Longitude coordinate for Mapbox tileset

ChartType

Top 200, Viral 50, Top 50. Allows ChartEntry rows to be filtered and grouped by chart type without repeated strings.

Column	Type	Constraints	Description
chart_type_id	INT	PK, IDENTITY	Auto-incremented surrogate primary key
name	NVARCHAR(50)	NOT NULL, UNIQUE	Chart name: 'Top 200', 'Viral 50', or 'Top 50'
rank_start	SMALLINT	NOT NULL	Lowest rank number on this chart (always 1)
rank_end	SMALLINT	NOT NULL	Highest rank number on this chart (50, 200, etc.)

Core Entity Tables

Artist

Column	Type	Constraints	Description
artist_id	INT	PK, IDENTITY	Auto-incremented surrogate primary key
name	NVARCHAR (255)	NOT NULL, UNIQUE	Artist display name as it appears in the source datasets

Album

Optional — Dataset 1 has no album data. album_id is nullable on Song.

Column	Type	Constraints	Description
album_id	INT	PK, IDENTITY	Auto-incremented surrogate primary key
name	NVARCHAR (255)	NOT NULL	Album title from Dataset 2
release_date	DATE	NULL	Album release date from Dataset 2. Format may vary or be missing.

Song

One row per unique track. spotify_id is the cross-dataset matching key — parsed from URL in Dataset 1, native in Dataset 2.

Column	Type	Constraints	Description
song_id	INT	PK, IDENTITY	Auto-incremented surrogate primary key
spotify_id	NVARCHAR (50)	NULL, UNIQUE	Spotify track ID. Parsed from URL in Dataset 1; native in Dataset 2. Nullable where parsing fails.
title	NVARCHAR (255)	NOT NULL	Song title as it appears in the source dataset
duration_ms	INT	NULL	Duration in milliseconds. Dataset 2 only.
is_explicit	BIT	NULL	Explicit lyrics flag. Dataset 2 only.
album_id	INT	NULL, FK->Album	Reference to album. Nullable — no album data in Dataset 1.

ArtistSong

Resolves many-to-many between Artist and Song. is_primary flag distinguishes primary vs. featured artists — important for Vinyl card display logic.

Column	Type	Constraints	Description
artist_song_id	INT	PK, IDENTITY	Auto-incremented surrogate primary key
artist_id	INT	NOT NULL, FK->Artist	Reference to the credited artist
song_id	INT	NOT NULL, FK->Song	Reference to the song

Column	Type	Constraints	Description
is_primary	BIT	NOT NULL, DEFAULT 1	True = primary credited artist. False = featured artist.

AudioFeatures

Stored separately from Song to signal that audio features are secondary, reach-goal data — not required for any MVP feature. UNIQUE constraint on song_id enforces one-to-one relationship. All feature columns nullable to handle incomplete rows.

Column	Type	Constraints	Description
audio_features_id	INT	PK, IDENTITY	Auto-incremented surrogate primary key
song_id	INT	NOT NULL, UNIQUE, FK->Song	One-to-one with Song. UNIQUE enforces relationship.
danceability	FLOAT	NULL	Suitability for dancing. Range 0.0-1.0.
energy	FLOAT	NULL	Intensity and activity level. Range 0.0-1.0.
valence	FLOAT	NULL	Musical positiveness. High = happy, low = sad. Range 0.0-1.0.
tempo	FLOAT	NULL	Estimated tempo in BPM.
speechiness	FLOAT	NULL	Presence of spoken words. Range 0.0-1.0.
acousticness	FLOAT	NULL	Acoustic quality confidence. Range 0.0-1.0.
instrumentalness	FLOAT	NULL	Likelihood of no vocals. >0.5 = likely instrumental. Range 0.0-1.0.
liveness	FLOAT	NULL	Live audience presence. >0.8 = strong likelihood of live. Range 0.0-1.0.
loudness	FLOAT	NULL	Overall loudness in dB. Typical range -60 to 0.
key	TINYINT	NULL	Pitch class notation (0=C, 1=C#, ... 11=B). -1 if no key detected.
mode	TINYINT	NULL	1 = major, 0 = minor.
time_signature	TINYINT	NULL	Estimated beats per bar. Range 3-7.

ChartEntry — The Core Fact Table

Largest table in the schema. Every row = one song on one chart in one country on one date. Estimated 13–16 million rows. All stored procedures query this table — indexing is critical.

Column	Type	Constraints	Description
chart_entry_id	INT	PK, IDENTITY	Auto-incremented surrogate primary key
song_id	INT	NOT NULL, FK->Song	Reference to the charting song
country_id	INT	NULL, FK->Country	Reference to country. NULL = Global Top 50 (Dataset 2 only).

Column	Type	Constraints	Description
chart_type_id	INT	NOT NULL, FK->ChartType	Reference to chart type
snapshot_date	DATE	NOT NULL	Date this chart entry was recorded
rank	SMALLINT	NOT NULL	Chart position on this date. Range depends on chart type.
streams	BIGINT	NULL	Raw stream count. Dataset 1 Top 200 only. NULL for Viral 50 and all Dataset 2.
popularity	TINYINT	NULL	Spotify proprietary score 0-100. Dataset 2 only. Not comparable to streams.
daily_movement	SMALLINT	NULL	Rank change vs. prior day (integer delta). Dataset 2 only.
weekly_movement	SMALLINT	NULL	Rank change vs. same day prior week (integer delta). Dataset 2 only.
trend	NVARCHAR(20)	NULL	Trend string from Dataset 1: MOVE_UP, MOVE_DOWN, SAME, NEW_ENTRY. Not in Dataset 2.

Required indexes on ChartEntry (minimum):

- (country_id, snapshot_date) — for all country+date filtered queries
- (song_id, country_id) — for overlap and Hidden Gem calculations
- (song_id, snapshot_date) — for geographic spread / discovery gap calculations
- (chart_type_id) — for filtering by Top 200 vs Viral 50 vs Top 50

4. Database Performance Philosophy — Pre-Computation First

The app works with 13–16 million rows in ChartEntry alone. Processing concerns are a backend and database problem — not a frontend rendering problem. Solutions live in the data layer.

The guiding principle: anything the app needs to display that requires aggregation, ranking, or cross-country comparison must be pre-computed and stored — not derived at request time. The raw ChartEntry table is the source of truth that feeds population procedures. It is not what the API queries directly at runtime.

The two classes of stored procedure:

- **Population procedures** — expensive, run once on data load or on a schedule. NEVER called at request time. Do the heavy lifting against ChartEntry.
- **Read procedures** — fast, called by the API at request time. Query pre-computed summary tables only. Should almost never touch ChartEntry directly.

Pre-Computed Summary Tables to Build

- HiddenGems — most expensive derivation. For every country and year: which songs chart in N+ other countries but not here. Pre-populated once, fast indexed read at runtime.
- CountryYearStats — one row per country per year: total charted, shared count, unique count, overlap %. Feeds KPI displays on country profile, comparison page, and dashboard.
- SongCountryPresence — for each song and year, how many countries it charted in. Foundation of Hidden Gems logic and minimum country count filter.
- GlobalOverlapByYear — year-by-year (or month-by-month) global overlap percentage. Feeds the dashboard trend line. Pre-populated = tiny SELECT at runtime instead of full table aggregation.
- DiscoveryGapByDay — for each song: origin country, spread country, and days between first chart appearances. Feeds KPI 2 (Average Discovery Gap) and the Discovery Gap Distribution histogram.
- IsolationScoreByCountry — per country per year: local song count, shared song count, isolation % score. Feeds KPI 3 (Most Isolated Region) and the Regional Isolation Scores chart. Apply minimum 100 ChartEntry threshold to exclude countries with too few entries.
- PeakReachBySong — for each song and date: count of distinct countries charted simultaneously. Feeds KPI 4 (Peak Cross-Regional Reach).
- TrendVelocityBySong — pre-computed trend velocity per song per country: uses daily_movement integers from Dataset 2, normalized +1/0/-1 from trend strings in Dataset 1. Never derived live.

HiddenGems table structure:

HiddenGemID	INT	IDENTITY	PRIMARY KEY
CountryID	INT	NOT NULL	FK -> Country
SongID	INT	NOT NULL	FK -> Song
ChartYear	INT	NOT NULL	
CountriesCharting	INT	NOT NULL	-- how many other countries it charts in
TrendScore	DECIMAL(10,4)		-- formula defined by BDA team
FOREIGN KEY (CountryID) REFERENCES Country(country_id)			
FOREIGN KEY (SongID) REFERENCES Song(song_id)			

5. KPI Definitions — Global Overlap Dashboard

Four headline KPIs are displayed prominently at the top of the dashboard before any trend charts. Each is a single number that directly proves the Discovery Gap is real, measurable, and consistent. All KPIs are calculated via stored procedures and are filterable by date range.

This is a historical analysis, not a real-time prediction tool. KPIs reflect patterns observed across the dataset. The two dataset periods (2017–2021 and 2023–2025) must not be averaged together without clearly labeling them as distinct timeframes.

KPI 1 — Global Overlap Rate

What it measures: of all songs that charted across any country during the dataset period, what percentage appeared in 2+ countries' charts?

Why it matters: the headline number. A low overlap rate is direct evidence that music discovery is regionally siloed.

Formula:

```
Global Overlap Rate (%) =
  (Songs charting in 2+ countries / Total unique charting songs) * 100
```

SP Inputs: date range (defaults to full dataset)

SP Outputs: overlap_pct, total_unique_songs, songs_in_2plus_countries

Display: bold percentage with trend indicator (up/down vs. prior period)

Recharts component: single stat display — no chart, just the number. Trend badge below.

Note: calculated separately per dataset period. Never average the two periods together.

KPI 2 — Average Discovery Gap (Days)

What it measures: when a song first appears on any country's chart, how many days on average before it appears on a second country's chart?

Why it matters: the most compelling KPI. Makes the abstract problem concrete — quantifies the literal lag between a song gaining momentum somewhere and that momentum crossing a border.

Formula:

```
For each song:
  first_chart_date    = MIN(snapshot_date) across all countries
  second_country_date = MIN(snapshot_date) WHERE country_id != first_country_id
  discovery_gap_days  = second_country_date - first_chart_date

Average Discovery Gap = AVG(discovery_gap_days)
  across all songs that charted in 2+ countries
```

SP Inputs: date range, minimum country count threshold (default: 2)

SP Outputs: avg_gap_days, median_gap_days, sample_size

Display: bold number of days. Show median alongside average — outliers skew the mean significantly.

Recharts component: single stat display for the KPI card. Histogram (bar chart) for the Discovery Gap Distribution chart below — bucket songs by gap length: 0-7d, 8-14d, 15-30d, 31-60d, 61-90d, 90d+.

Feeds pre-computed table: DiscoveryGapByDay — run at population time, never at request time.

Note: songs that only ever charted in one country are excluded — no second country date exists.

KPI 3 — Most Globally Isolated Region

What it measures: which country has the lowest overlap with global chart trends — meaning the highest percentage of its charting songs appear nowhere else.

Why it matters: gives the geographic dimension a headline. Identifies where the Discovery Gap is largest and gives users a point of interest to explore further on the globe.

Formula:


```
For each country:
  local_songs = COUNT(DISTINCT song_id) WHERE country_id = @country
  shared_songs = COUNT(DISTINCT song_id) WHERE country_id = @country
                AND song_id IN (songs charting in any other country)

isolation_score (%) =
  ((local_songs - shared_songs) / local_songs) * 100

Most Isolated = Country with highest isolation_score
WHERE total chart entries >= 100 (minimum threshold)
```

SP Inputs: date range

SP Outputs: ranked list of all qualifying countries by isolation_score. Dashboard shows top result as the KPI; full ranked list feeds the Regional Isolation Scores chart.

Display: country name + isolation score % (e.g. "Indonesia — 84% locally unique")

Recharts component: single stat display for KPI card. Horizontal bar chart (BarChart with layout='vertical') for Regional Isolation Scores — color-coded by isolation level: high = red (#f87171), mid = orange (#fb923c), low = green (#4ade80).

Feeds pre-computed table: IsolationScoreByCountry. Minimum 100 ChartEntry threshold enforced at population time.

Open question: show most isolated AND least isolated? Showing both tells a more complete story.

KPI 4 — Peak Cross-Regional Reach

What it measures: the highest number of countries a single song simultaneously charted in at any point in the dataset.

Why it matters: the ceiling number. Makes the Discovery Gap meaningful by contrast — if the peak is 62 countries and the average is 3, that gap is the story.

Formula:

```
For each song, for each snapshot_date:
  country_count = COUNT(DISTINCT country_id)
                  WHERE song_id = @song AND snapshot_date = @date

Peak Reach = MAX(country_count) across all songs and all dates
```

SP Inputs: date range

SP Outputs: peak_country_count, song_id, song_title, artist_name, peak_date

Display: number of countries + song title + artist name + date. Rendered as a Vinyl card component inline on the dashboard — not plain text.

Recharts component: no chart for this KPI card itself. The Vinyl card IS the display. The Global Reach Over Time bar chart below (avg countries per song per year) provides the trend context for this number.

Feeds pre-computed table: PeakReachBySong. Run at population time.

Open KPI Questions for Team

- Should KPIs be filterable by date range in the UI, or static across the full dataset? This directly affects SP signature design — decide before writing procs.
- KPI 3: show most isolated only, or most isolated AND least isolated side by side? Both tells a more complete story.
- KPI 4: Vinyl card component confirmed for display — Eli to implement.

6. Recharts — Dashboard Chart Components

The charting library candidate is Recharts (vs. Victory Native — final selection pending evaluation of React Native compatibility and rendering performance). All chart component design below assumes Recharts. If Victory Native is selected, the data shape contracts remain the same — only the component API changes.

Build one shared chart component per chart type that accepts different data shapes via props. The same overlap line chart should work for the global dashboard, the country profile page, and the comparison page — just with different data passed in. Do not rebuild the same chart three times.

Chart 1 — Global Overlap Rate Over Time (Line Chart)

Recharts component: LineChart with Area fill

Data shape the SP must return:

```
-- One row per year (or month if monthly granularity chosen)
SELECT
  period_label,      -- '2017', '2018', etc. (string for X axis label)
  period_year,       -- INT for sorting
  period_month,      -- INT NULL (NULL if yearly granularity)
  overlap_pct,       -- DECIMAL: % of songs in 2+ countries
  total_unique_songs, -- INT
  songs_in_2plus,    -- INT
  is_gap             -- BIT: 1 = this point falls in the 22-month data gap
                    -- used to render dashed line segment in Recharts
FROM GlobalOverlapByYear
ORDER BY period_year, period_month
```

Recharts implementation notes:

- Use is_gap = 1 rows to render a dashed line segment across the gap (pass null for overlap_pct on gap points, let Recharts connectNulls handle the visual break or render a custom dashed segment)
- Add a ReferenceArea spanning the gap dates to show the shaded gap region with a 'Data Gap (22 months)' label
- X axis: period_label. Y axis: overlap_pct formatted as percentage.
- This component is reused on Country Profile and Comparison pages — just feed it country-scoped data instead of global data

Chart 2 — Regional Isolation Scores (Horizontal Bar Chart)

Recharts component: BarChart with layout='vertical'

Data shape the SP must return:

```
-- Ranked list of all qualifying countries by isolation score
SELECT TOP 20
  country_name,      -- NVARCHAR: display label on Y axis
  iso_code,          -- CHAR(2): for navigation if user clicks a bar
  isolation_score,    -- DECIMAL: % locally unique songs (X axis value)
  isolation_tier      -- NVARCHAR: 'high' | 'mid' | 'low' for color coding
                    --   high > 65%, mid 40-65%, low < 40%
FROM IsolationScoreByCountry
WHERE total_entries >= 100
ORDER BY isolation_score DESC
```

Recharts implementation notes:

- Color bars dynamically based on isolation_tier: high = #f87171, mid = #fb923c, low = #4ade80
- X axis: 0-100%. Y axis: country_name. Use a custom Cell per bar to apply per-bar color.
- Consider making bars clickable — on click, navigate to that country's profile page using iso_code

Chart 3 — Discovery Gap Distribution (Histogram / Bar Chart)

Recharts component: BarChart with pre-bucketed data

Data shape the SP must return:

```
-- Pre-bucketed distribution — bucketing done in the SP, not in JS
SELECT
  bucket_label,      -- NVARCHAR: '0-7d', '8-14d', '15-30d', '31-60d', '61-90d',
                    -- '90d+'
  bucket_order,      -- INT: for correct sort order (1-6)
  song_count         -- INT: number of songs falling in this bucket
FROM DiscoveryGapByDay
-- Bucketing logic lives in the SP population proc, not at read time
ORDER BY bucket_order
```

Recharts implementation notes:

- Standard vertical BarChart. X axis: bucket_label. Y axis: song_count formatted as 'Xk'.
- Single color for all bars: #818cf8 (indigo) at 0.7 opacity
- Buckets are pre-computed — no client-side grouping or math needed

Chart 4 — Global Reach Over Time (Bar Chart)

Recharts component: BarChart (vertical bars, one per year)

Data shape the SP must return:

```
-- One row per year, skipping gap years
SELECT
  period_year,      -- INT: X axis label
  avg_countries,    -- DECIMAL(4,2): average countries per charting song
  is_gap           -- BIT: exclude gap years from render
FROM GlobalOverlapByYear
WHERE is_gap = 0
ORDER BY period_year
```

Recharts implementation notes:

- Single color bars: #34d399 (emerald) at 0.7 opacity
- Y axis starts at 0. X axis: year as string.
- Omit gap years entirely rather than showing empty bars — cleaner than the line chart treatment for bar charts

Shared KPI Stat Card (No Chart)

KPI 1, KPI 2, KPI 3, and the number portion of KPI 4 are displayed as stat cards — bold headline number with supporting label and badge. These are custom-styled div components, not Recharts. KPI 4's Vinyl card is its own existing component.

Data contracts for the KPI stat cards:

```
-- KPI 1: sp_GetGlobalOverlapRate
overlap_pct DECIMAL, total_unique_songs INT, songs_in_2plus INT

-- KPI 2: sp_GetAverageDiscoveryGap
avg_gap_days INT, median_gap_days INT, sample_size INT

-- KPI 3: sp_GetIsolationLeader
country_name NVARCHAR, iso_code CHAR(2), isolation_score DECIMAL

-- KPI 4: sp_GetPeakCrossRegionalReach
peak_country_count INT, song_title NVARCHAR, artist_name NVARCHAR, peak_date
DATE
```

Shared KPIs at different scopes — reuse the same components:

- Global Overlap Dashboard: global scope, full date range
- Country Profile Page: single-country scope, year-scoped
- Country Comparison Page: two-country scope, year-scoped
- The data shape differs per scope but the visual component is identical — feed it different data via props

7. All Stored Procedures

Procedures are scoped to screen-level interactions and pre-computed summary tables. The API never returns raw ChartEntry rows.

7a. Population / Maintenance Procedures

Run once on data load or on a schedule. NEVER called at request time. These are the expensive procedures — they run against ChartEntry directly.

- **sp_PopulateHiddenGems**
 - Finds songs charting in N+ countries but absent from a given country — most expensive operation
 - Computes TrendScore per hidden gem, populates HiddenGems summary table

- NEW_ENTRY trend value from Dataset 1 should be weighted as a positive signal in TrendScore formula
- **sp_PopulateCountryYearStats**
 - Aggregates per country per year: total charted, shared count, unique count, overlap %
 - Feeds KPI displays on country profile, comparison page, and dashboard
- **sp_PopulateSongCountryPresence**
 - Per song per year: COUNT(DISTINCT country_id) across ChartEntry
 - Foundation of Hidden Gems logic and minimum country count filter
- **sp_PopulateGlobalOverlapByYear**
 - Year-by-year (or month-by-month) global overlap percentage + avg countries per song
 - Includes is_gap BIT flag for the 22-month data gap — feeds chart rendering logic in Recharts
 - Decide on yearly vs. monthly granularity BEFORE writing this proc — it affects SP signature, table structure, and Recharts X axis
- **sp_PopulateDiscoveryGapByDay**
 - For each song: origin country (first chart date), spread country, days between appearances
 - Feeds KPI 2 (Average Discovery Gap) and the Discovery Gap Distribution histogram
 - Most complex date calculation — compare MIN(snapshot_date) per country per song across full ChartEntry
 - Pre-bucket into gap bands (0-7d, 8-14d, 15-30d, 31-60d, 61-90d, 90d+) at population time — do NOT bucket in JS
- **sp_PopulateIsolationScoreByCountry**
 - Per country: local_songs, shared_songs, isolation_score %
 - Apply minimum 100 ChartEntry threshold — countries below threshold are excluded
 - Assign isolation_tier: 'high' (>65%), 'mid' (40-65%), 'low' (<40%) — used for Recharts bar coloring
 - Feeds KPI 3 display and Regional Isolation Scores horizontal bar chart
- **sp_PopulatePeakReachBySong**
 - For each song per snapshot_date: COUNT(DISTINCT country_id)
 - Stores MAX(country_count) per song with the date it occurred
 - Feeds KPI 4 (Peak Cross-Regional Reach) — song title and artist name joined at population time
- **sp_PopulateTrendVelocityBySong**
 - Per song per country: normalize Dataset 1 trend strings (MOVE_UP=+1, MOVE_DOWN=-1, SAME=0, NEW_ENTRY=signal)
 - For Dataset 2: use daily_movement and weekly_movement integers directly
 - Compute rolling average velocity over a defined window (e.g. 7 days or 4 snapshots)
 - Store normalized TrendVelocity value — never derive live from ChartEntry

- **sp_PopulateOverlapSummary (potential)**
 - Pre-computes overlap % by country pair and year
 - Evaluate after initial build — only needed if live comparison computation is too slow

7b. Read Procedures — Globe / Discovery Screen

- **sp_GetGlobeSummary(@Year)**
 - One row per country in the dataset — lightweight
 - Returns: country_code, country_name, lat, long, hidden_gem_count, top_album_name
 - Reads from HiddenGems and CountryYearStats pre-computed tables — no ChartEntry access
 - Powers globe dots and country list sidebar. Hidden gem count per dot makes the gap immediately visible.
 - Also feeds Mapbox tileset export — same data, different format

7c. Read Procedures — Country Profile Page

- **sp_GetCountryProfile(@CountryCode, @Year)**
 - Returns summary stats from CountryYearStats: total charted, shared count, unique count, overlap %
 - Returns top shared songs and top unique songs — reads pre-computed tables, not ChartEntry
- **sp_GetCountryHiddenGemsPreview(@CountryCode, @Year)**
 - Top 5 hidden gems for the teaser widget on the country page — fast read from HiddenGems table
 - Ordered by TrendScore DESC

7d. Read Procedures — Hidden Gems Screen

- **sp_GetHiddenGems(@CountryCode, @Year, @MinCountries, @Offset, @PageSize)**
 - Full paginated hidden gems list — pagination params required from the start
 - Returns: song_name, album_name, artist_name, genre (if available), preview_url, trend_score, countries_charting_count
 - Fast SELECT against pre-computed HiddenGems table only

7e. Read Procedures — Country Comparison Page

- **sp_GetCountryComparison(@CountryCodeA, @CountryCodeB, @Year)**
 - Overlap KPIs from pre-computed CountryYearStats
 - Shared songs list with rank in each country side by side
 - Unique to Country A and unique to Country B lists
- **sp_GetComparisonHiddenGems(@CountryCodeA, @CountryCodeB, @Year)**
 - Songs trending in surrounding regions absent from BOTH selected countries

- Most powerful data point on this page — the gap both markets share
- Reads from HiddenGems filtered to exclude both country IDs

7f. Read Procedures — Global Overlap Dashboard

- **sp_GetGlobalOverlapRate(@DateStart, @DateEnd)**
 - Returns: overlap_pct, total_unique_songs, songs_in_2plus_countries
 - Feeds KPI 1 stat card
- **sp_GetAverageDiscoveryGap(@DateStart, @DateEnd, @MinCountries = 2)**
 - Returns: avg_gap_days, median_gap_days, sample_size
 - Reads from DiscoveryGapByDay pre-computed table
 - Feeds KPI 2 stat card
- **sp_GetDiscoveryGapDistribution(@DateStart, @DateEnd)**
 - Returns pre-bucketed histogram data: bucket_label, bucket_order, song_count
 - Buckets: 0-7d, 8-14d, 15-30d, 31-60d, 61-90d, 90d+
 - Reads from DiscoveryGapByDay — no bucketing in JS
 - Feeds Discovery Gap Distribution histogram (Recharts BarChart)
- **sp_GetIsolationLeader(@DateStart, @DateEnd)**
 - Returns: country_name, iso_code, isolation_score for the top result (KPI 3 card)
- **sp_GetIsolationRanking(@DateStart, @DateEnd)**
 - Returns ranked list of all qualifying countries: country_name, iso_code, isolation_score, isolation_tier
 - Feeds Regional Isolation Scores horizontal bar chart (Recharts BarChart layout='vertical')
 - Separated from sp_GetIsolationLeader so the KPI card and the chart can be fetched independently
- **sp_GetPeakCrossRegionalReach(@DateStart, @DateEnd)**
 - Returns: peak_country_count, song_title, artist_name, peak_date
 - Reads from PeakReachBySong pre-computed table
 - Feeds KPI 4 stat card + Vinyl card display
- **sp_GetGlobalOverlapTrend(@DateStart, @DateEnd)**
 - Returns one row per year (or month): period_label, period_year, period_month, overlap_pct, avg_countries, is_gap
 - is_gap BIT is critical — Recharts uses it to render the dashed segment and ReferenceArea for the 22-month gap
 - Takes a DATE RANGE, not a single year — design proc signature and API DTO accordingly
 - Feeds both the Global Overlap Rate Over Time line chart AND the Global Reach Over Time bar chart

8. Mapbox Tileset & Country Lookup Table

The Country lookup table already being built is the spine of the Mapbox tileset. No significant extra work is needed — the tileset is a shaped export of data already being built.

The architecture split:

- **Mapbox tileset = globe rendering and hover cards ONLY**
- **Normal .NET API = everything else: country detail, comparisons, hidden gems, dashboard**
- Mapbox is the map rendering layer only — not a database replacement

What the tileset needs from the Country table:

- country_id (stable identifier), country_name, iso_code, latitude, longitude
- Lightweight pre-aggregated summary fields for hover cards: hidden_gem_count (from HiddenGems), top_album_name (from CountryYearStats)
- The region column enables the region filter on the globe screen — add during schema finalization

Platform note:

- Web: Mapbox GL JS. Mobile: React Native Mapbox. Same underlying data strategy and tileset across both platforms — different frontend implementations only.

Fallback plan:

- If Mapbox causes too much friction: switch to MapLibre. Data strategy and tileset export are identical. Discovery Globe becomes Discovery Map.

Coordinate with Eli on exact tileset fields and export format so globe rendering does not require a separate data preparation step.

9. Filters

Filters are not data-narrowing tools. Every filter is a way for the user to ask a specific version of the discovery gap question. UI framing should reflect this.

9a. Globe / Filtering Screen

Confirmed — build from guaranteed data:

- **Year / year range — most critical filter. The gap story lives in time. Data gap (Dec 2021–Oct 2023) must be visually marked.**
- Region — filter globe to a geographic region. Reads from region column on Country table (add during schema finalization).
- Chart type — Top 200, Viral 50, Top 50. Reads ChartType lookup table. Important: Viral 50 is the best signal for early-stage gap detection.

- Minimum country count — most direct discovery gap filter. High global presence + absent from a market = strongest Hidden Gem candidates.
- Chart position threshold — Top 10, Top 25, Top 50. Focuses the gap on genuine crossover hits.
- Artist search / filter — where is this artist huge, and where have they not broken through yet.

Stretch goals — only if external API or Dataset 2 coverage confirms availability:

- Genre — not in either dataset natively. Requires external API enrichment. Do not design core filter architecture around this.
- Audio features (danceability, valence, energy, tempo) — AVAILABLE in Dataset 2 already. Could power mood-based filters as a reach goal without any external API.

9b. Preselected Filter Presets

Each preset is a named version of the discovery gap — a story, not just a shortcut:

- "Global Hits You Might Have Missed" — high country count, Top 10 chart position, most recent year
- "The Viral Wave" — chart type: Viral 50, high country count, any year. Surfaces early-stage crossover songs before they hit the mainstream charts.
- "Your Region vs. The World" — selected region highlighted, shows what is trending everywhere else but missing locally
- "One-Country Wonders" — minimum country count = 1. Songs charting nowhere else — hyper-local music that never crossed over
- "The Biggest Crossover Years" — year range set to statistically highest global overlap years from the data
- "Latin Music's Global Takeover" — genre filter: STRETCH GOAL. Region: Latin America + worldwide reach filter as fallback.

9c. Country Profile Page Filters

- Year selector / dropdown — required. Year change triggers loading state. Decide now so all proc signatures are built around it.
- Chart type (Top 200 vs Viral 50 vs Top 50)
- Minimum country count threshold — for Hidden Gems preview teaser
- Chart position threshold — for charted songs list

9d. Country Comparison Page Filters

- Year selector / dropdown — required, same loading state behavior
- Chart type filter
- Chart position threshold
- Minimum country count — for the absent songs / comparison hidden gems section
- Nmn

11. Trend Tracking — Backend Approach

Three distinct things 'trend over time' can mean in this app. Each requires a different approach.

Trend Type 1 — Chart Position Trajectory (Song Level)

How a song's rank moves day by day within a single country. Powers Hidden Gem scoring and the trend indicator on song cards.

- Dataset 2: use `daily_movement` (integer delta) and `weekly_movement` (integer delta) directly
- Dataset 1: normalize trend string — `MOVE_UP` = +1, `MOVE_DOWN` = -1, `SAME` = 0, `NEW_ENTRY` = +1 signal
- Pre-compute `TrendVelocity` as a rolling average over a defined window — store in `TrendVelocityBySong`
- Never derive `TrendVelocity` live from `ChartEntry` at request time

Trend Type 2 — Geographic Spread Over Time (Song Level)

How a song moves from country to country — which country charted it first, and how many days until each subsequent country. This is the core of KPI 2 and the Discovery Gap Distribution chart.

- Pre-compute in `sp_PopulateDiscoveryGapByDay` — expensive date comparison across `ChartEntry`
- Store `origin_country_id`, `spread_country_id`, `days_to_spread` per song
- From this table: derive average days to first border crossing, geographic spread sequence, and histogram distribution

Trend Type 3 — Macro Overlap Trends (Global Level)

Year-over-year or month-over-month story of how globally connected music markets are. Powers the dashboard trend line.

- Decision needed: yearly or monthly granularity? Monthly = smoother chart, more rows, more complex population proc. Yearly = simpler, probably sufficient for capstone.
- This decision affects SP signature (`@YearStart/@YearEnd` vs `@DateStart/@DateEnd`), table structure, and Recharts X axis labels — decide before writing the population proc.
- Pre-compute in `sp_PopulateGlobalOverlapByYear` — store `is_gap` BIT for each period

12. Open Items

- **Recharts vs. Victory Native** — confirm final charting library selection before writing any chart components. All data shape contracts in Section 6 assume Recharts but remain valid for Victory Native.
- **External API plan** — audio features are already in Dataset 2 and do NOT require an external API. The only remaining gap is genre tags. Come back with a concrete plan only if genre filtering is confirmed as a target.

- **Discovery gap framing in UI copy** — the gap must be in the copy, not just the data. Splash screen, filter preset names, empty states, hover card labels, chart titles — all of it should speak the language of the gap.
- **Comparison page data flow** — two country states simultaneously, potentially two parallel API calls plus the combined comparison proc. Not the same as a single country page. Design component state accordingly.
- **Year selector dropdown** — needed on Country Profile, Hidden Gems, and Comparison pages. Year change triggers a loading state. This is a concrete decision needed now so all proc signatures are built around it.
- **Tileset coordination** — the Country table Leena is already building is the Mapbox tileset foundation. Coordinate on exact fields and export format before globe build begins.
- **Shared chart components** — build one Recharts component per chart type that accepts different data via props (global scope, country scope, two-country scope). Do not rebuild the same chart three times.
- **Recharts data gap rendering** — the `is_gap` BIT from `sp_GetGlobalOverlapTrend` is what drives the dashed line segment and `ReferenceArea` on the overlap trend chart. Handle null `overlap_pct` values and use `connectNulls` or custom segment rendering to show the gap visually.
- **Backend processing context** — when Leena flags concerns about data size and processing time, these are backend and database concerns. Solutions live in pre-computed summary tables and indexed reads — not in UI optimizations.

Living document — update as decisions are made. Last updated April 2026.